

Golden Dataset API

Coder Review & Correction endpoints — application and Postman guide

Every time a human coder reviews predicted ICD/CPT codes and submits ACCEPT / EDIT / DELETE / ADD actions, the system writes the result to the **coder_corrections** table in Postgres and to the **correction_memory** collection in Qdrant. Together these form the *golden dataset*: a growing, vetted record of how real coders correct the pipeline. The dataset is read back automatically on every new report (via the pre-Agent-3 correction_memory RAG step) so the model learns from prior corrections.

This document covers the four endpoints used to populate that dataset, how to call them from an application, and how to drive them from Postman.

At a glance

Concern	Detail
Service	Gateway → Orchestrator (review router)
Base URL	http://<host>:7500 (GATEWAY_PORT, UAT default)
Auth	JWT Bearer token — issued by POST /auth/token
Content-Type	application/json
Rate limiting	Per-client daily limit (enforced by Gateway)
Side effects	Postgres coder_corrections + Qdrant correction_memory upsert
Report status	Moves to APPROVED after a successful submit

1. Authentication

The JWT is per-application, not per-coder. Read this carefully — it is the most commonly misread part of this API.

- The token is minted **once per hospital client** by an operator using the X-Admin-Secret header. Coders never call /auth/token themselves.
- The JWT payload carries client_id + hospital_name + daily_limit only. It does **not** carry a coder identity.
- The same JWT authorizes **every** /api/* endpoint — upload, reports, encounters, review, rules, billing. If the application already holds a JWT for the upload/report APIs, **reuse it for the review (golden-dataset) APIs**. Do not mint a new one.
- The reviewing coder's identity is passed in the **request body** as coder_id (a User UUID from the users table) on each submit — not in the token. The application is responsible for tracking which coder is logged in and forwarding their UUID.
- Rate limiting and audit are keyed on client_id from the JWT; per-coder attribution is recorded on the coder_corrections and reviewed_codes rows via coder_id.

On every authenticated call, send: Authorization: Bearer <token>.

1.1 POST /auth/token (operator-only, once per client)

Issues a JWT for a hospital client. Requires the shared X-Admin-Secret header — this is operator-only and must never ship to an end-user app or be exposed to coders. **Call this once per hospital client** (e.g., during onboarding), store the returned token in the application, and reuse it for all subsequent /api/* calls including the

review endpoints. The application does **not** call `/auth/token` per login or per coder.

```
POST /auth/token
Host: <gateway-host>:7500
Content-Type: application/json
X-Admin-Secret: <ADMIN_SECRET>

{
  "client_id": "hospital-acme",
  "hospital_name": "Acme General",
  "daily_limit": 500
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "client_id": "hospital-acme",
  "hospital_name": "Acme General",
  "daily_limit": 500,
  "expires_days": 30
}
```

1.2 Coder registration — `POST /admin/users`

Every *submit* call must include `coder_id` — a UUID from the orchestrator's users table. The application obtains that UUID by calling `POST /admin/users` once per coder. This is the **only** supported registration path: the application does not (and must not) touch the orchestrator database directly.

Gated by JWT, same as every `/api/*` endpoint. The application uses its existing hospital JWT (the one issued by `/auth/token` at onboarding) — no separate admin secret is needed for coder registration. The admin secret is reserved for `/auth/token` only.

```
POST /admin/users
Host: <gateway-host>:7500
Content-Type: application/json
Authorization: Bearer <token>

{
  "name": "Jane Doe",
  "email": "jane@hospital-a.example",
  "role": "CODER" // optional; CODER | ADMIN | VIEWER; defaults to CODER
}
```

Response:

```
201 Created
{
  "id": "8a3f1c10-...",
  "name": "Jane Doe",
  "email": "jane@hospital-a.example",
  "role": "CODER",
  "active": true,
  "created_at": "2026-05-14T18:22:11+00:00"
}
```

Store the returned `id` on your side (e.g., as `users.public_id`) and send it as `coder_id` on every `/api/review/*/submit`. The application is responsible for calling `/admin/users` at coder-onboarding time; the orchestrator does not create users on-demand from submit traffic.

Error responses

Status	When it happens
401	Missing / invalid / expired JWT.
409	Email already registered (see §1.4 — emails are globally unique).
422	name empty, email malformed, or role not in CODER ADMIN VIEWER.
503	Orchestrator unreachable from gateway.

1.3 Tenancy of the users table (important caveat)

The users table is **global, not scoped per client_id / hospital**:

- users has no client_id / hospital_id column.
- The orchestrator never sees the JWT's client_id; the gateway terminates auth and forwards the request body as-is.
- The only check on submit is `SELECT * FROM users WHERE id = :coder_id` — no cross-check that the coder belongs to the calling hospital.

Implication: a hospital-A JWT carrying a hospital-B coder_id will **succeed**, not 403. Per-hospital isolation is not enforced today; plan to add a hospital_id FK + a gateway-injected check when that becomes a requirement.

1.4 Email uniqueness (globally unique)

The users.email column has a **UNIQUE** constraint that is **global, not per-hospital**. A second POST /admin/users with an email that already exists returns HTTP 409.

Conventions to avoid collisions across hospitals:

- Prefer **hospital-scoped professional emails** (e.g., jane@hospital-a.example) — natural and unambiguous.
- If a coder only has a personal email, use the **RFC-5233 plus-tag trick**: jane+hospital-a@gmail.com — globally unique and still routes to jane@gmail.com.
- On 409, the application should decide whether to fail loudly or fetch the existing UUID via your own tracking (we do not currently expose a lookup-by-email endpoint).

Long-term fix: add users.client_id and switch uniqueness to (client_id, email). Blocked on the broader tenancy work described in §1.3.

1.3 Tenancy of the users table (important caveat)

The users table is **global, not scoped per client_id / hospital**:

- users has no client_id / hospital_id column.
- The orchestrator never sees the JWT's client_id; the gateway terminates auth and forwards the request body as-is.
- The only check on submit is `SELECT * FROM users WHERE id = :coder_id` — no cross-check that the coder belongs to the calling hospital.

Implication: a hospital-A JWT carrying a hospital-B coder_id will **succeed**, not 403. Global coder UUIDs work today; the API does not enforce per-hospital isolation. Plan to add a hospital_id FK + a gateway-injected check when that becomes a requirement.

2. Endpoint reference

Method	Path	Purpose
GET	/api/review/queue	List reports / encounters waiting for review, sorted by lowest avg confidence
GET	/api/review/report/{report_id}/codes	Predicted codes for a single report (single-report flow).
GET	/api/review/encounter/{encounter_id}/codes	Predicted codes for an encounter (multi-report flow).
POST	/api/review/report/{report_id}/submit	Submit ACCEPT / EDIT / DELETE / ADD actions for a report. Writes to go
POST	/api/review/encounter/{encounter_id}/submit	Same as above, for encounter-level codes.

2.1 GET /api/review/queue

Query params: page (default 1), page_size (default 20). Items are sorted by ascending average confidence so the least-certain predictions land at the top of the queue.

GET /api/review/queue?page=1&page_size=20
Authorization: Bearer <token>

Response 200:

```
{
  "items": [
    {
      "type": "report",
      "id": "8d3c...",
      "encounter_type": "OUTPATIENT",
      "avg_confidence": 0.612,
      "code_count": 7,
      "created_at": "2026-05-12T14:02:11Z",
      "report_type": "HP"
    },
    ...
  ],
  "total": 42,
  "page": 1,
  "page_size": 20
}
```

2.2 GET /api/review/report/{report_id}/codes

Returns every predicted code for the report, sorted by sequence_pos then by confidence descending. The id field on each item is the **predicted_code_id** you pass back in the submit payload.

GET /api/review/report/8d3c.../codes
Authorization: Bearer <token>

Response 200:

```
[
  {
    "id": "a7b2-...",
    "icd_code": "E11.9",
    "description": "Type 2 diabetes mellitus without complications",
    "confidence": 0.71,
    "code_type": "primary",
    "sequence_pos": 1,
    "evidence_json": { "source_chunks": [...] },
    "status": "PREDICTED"
  },
  ...
]
```

The encounter variant GET /api/review/encounter/{encounter_id}/codes behaves identically but scopes to a single encounter and filters out report-only codes.

2.3 POST /api/review/report/{report_id}/submit

This is the call that writes to the golden dataset. The payload is a list of action items plus the reviewing coder_id. Validation runs first, before any database writes — if any single item is invalid, the whole batch is rejected with HTTP 422.

POST /api/review/report/8d3c.../submit
Authorization: Bearer <token>
Content-Type: application/json

```
{
  "coder_id": "11111111-2222-3333-4444-555555555555",
  "actions": [
    {
      "predicted_code_id": "a7b2-...",
      "action": "ACCEPT"
    },
    {
      "predicted_code_id": "c91d-...",
      "action": "EDIT",
      "correct_code": "I10",
      "correct_description": "Essential (primary) hypertension",
      "reason": "Code in note says I10 not I11; HTN without heart involvement."
    },
    {
      "predicted_code_id": "fa30-...",
      "action": "DELETE",
      "reason": "Symptom code not separately reportable when underlying dx is coded."
    },
    {
      "action": "ADD",
      "correct_code": "Z79.4",
      "correct_description": "Long term (current) use of insulin",
      "code_type": "secondary",
      "sequence_pos": 3,
      "reason": "Patient on insulin per medication list – must be added per guidelines."
    }
  ]
}
```

Per-action validation rules

Action	Required fields	Notes
ACCEPT	predicted_code_id	No reason required. Writes a ReviewedCode row only — does not enter correction.
EDIT	predicted_code_id, correct_code, reason (≥20 chars)	correct_description optional but recommended. Writes correction.
DELETE	predicted_code_id, reason (≥20 chars)	Removes a predicted code. Writes correction.
ADD	correct_code, code_type, reason (≥20 chars)	code_type ∈ {primary, secondary, procedure, cpt}. sequence_pos optional.

reason must be at least 20 characters (after trim) for EDIT, DELETE, and ADD. This is enforced both at the API layer and in the correction writer — short or empty reasons are rejected with HTTP 422.

Response

Response 200:
{

```

"report_id": "8d3c-...",
"encounter_id": null,
"total_actions": 4,
"accepted": 1,
"edited": 1,
"deleted": 1,
"added": 1,
"corrections_written": 3,
"qdrant_sync_failures": 0,
"results": [
  { "predicted_code_id": "a7b2-...", "action": "ACCEPT", "success": true },
  { "predicted_code_id": "c91d-...", "action": "EDIT",
    "success": true, "correction_id": "f0...", "qdrant_synced": true },
  { "predicted_code_id": "fa30-...", "action": "DELETE",
    "success": true, "correction_id": "f1...", "qdrant_synced": true },
  { "predicted_code_id": "<new>", "action": "ADD",
    "success": true, "correction_id": "f2...", "qdrant_synced": true }
]
}

```

qdrant_sync_failures > 0 is non-fatal — the Postgres row is the source of truth, and a background job can re-sync to Qdrant later. **Report status** transitions PENDING_REVIEW → IN_REVIEW → APPROVED on success. A 409 is returned if the report is already APPROVED.

Common error responses

Status	When it happens
401	Missing / invalid / expired JWT.
404	report_id, encounter_id, or coder_id not found.
409	Report is already APPROVED, or pipeline is not yet complete.
422	Validation failed (reason too short, missing correct_code, bad action, etc.).
429	Daily rate limit exceeded for the client.
503	Orchestrator unreachable from gateway.

2.4 POST /api/review/encounter/{encounter_id}/submit

Identical payload and validation; the path differs and encounter_id on the response is populated instead of report_id. Use this when codes were produced at the encounter (multi-report) level.

3. Using the API from an application

Typical UI flow for a coder review screen:

1. Call GET `/api/review/queue` on screen load. Render the list and let the coder pick a report or encounter.
2. Call GET `/api/review/report/{id}/codes` (or the encounter variant) to populate the review table. Show `icd_code`, `description`, `confidence`, and `evidence_json`.
3. As the coder marks each row `ACCEPT` / `EDIT` / `DELETE` or adds a new code, build up an actions array. Keep the `predicted_code_id` from step 2 on every existing row.
4. On *Submit*, POST the batch to `/api/review/report/{id}/submit`. Display `corrections_written` back to the coder and flag any `qdrant_sync_failures`.

3.1 JavaScript (fetch) example

```
const BASE = "http://gateway.example.com:7500";
const TOKEN = localStorage.getItem("apiToken"); // set once at login

async function getQueue() {
  const r = await fetch(`${BASE}/api/review/queue?page=1&page_size=20`, {
    headers: { Authorization: `Bearer ${TOKEN}` },
  });
  if (!r.ok) throw new Error(`queue ${r.status}`);
  return r.json();
}

async function getReportCodes(reportId) {
  const r = await fetch(`${BASE}/api/review/report/${reportId}/codes`, {
    headers: { Authorization: `Bearer ${TOKEN}` },
  });
  return r.json();
}

async function submitReview(reportId, coderId, actions) {
  const r = await fetch(`${BASE}/api/review/report/${reportId}/submit`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${TOKEN}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ coder_id: coderId, actions }),
  });
  if (r.status === 422) {
    throw new Error("Validation failed: " + (await r.text()));
  }
  if (!r.ok) throw new Error(`submit ${r.status}`);
  return r.json();
}
```

3.2 Python (requests) example

```
import requests

BASE = "http://gateway.example.com:7500"
TOKEN = "<paste-jwt-here>"
HDR = {"Authorization": f"Bearer {TOKEN}"}

def submit_review(report_id, coder_id, actions):
    r = requests.post(
        f"{BASE}/api/review/report/{report_id}/submit",
        headers={**HDR, "Content-Type": "application/json"},
```

```

        json={"coder_id": coder_id, "actions": actions},
        timeout=30,
    )
    r.raise_for_status()
    return r.json()

actions = [
    {"predicted_code_id": "a7b2-...", "action": "ACCEPT"},
    {"predicted_code_id": "c91d-...", "action": "EDIT",
     "correct_code": "I10",
     "correct_description": "Essential (primary) hypertension",
     "reason": "Code in note says I10 not I11; HTN without heart involvement."},
]

print(submit_review("8d3c-...", "11111111-2222-3333-4444-555555555555", actions))

```

3.3 Application-side rules of thumb

- **Cache the JWT**; do not call /auth/token per request.
- **Don't silently swallow validation errors** — surface the 422 detail string to the coder so they can lengthen the reason or fill the missing field.
- **Never include PHI in the reason field.** It is embedded into Qdrant for similarity search and is not encrypted at the vector layer.
- **Submit once.** Each successful submit moves the report to APPROVED; a retry will return 409. Disable the button after first response.
- **Treat qdrant_sync_failures > 0 as a soft warning**, not an error — corrections are still saved in Postgres and will sync later.

4. Using the API in Postman

A ready-to-import collection ships alongside this PDF at docs/golden_dataset_api.postman_collection.json. The steps below describe how to set it up from scratch if you prefer.

Note on Postman vs. the real application. In Postman you will mint a JWT yourself (Request 1) because you are simulating the application. In production, this step happens **once per hospital client** at onboarding and the application reuses the same token for every `/api/*` call — coders do not log in to get a JWT.

4.1 Create an environment

In Postman, create a new **Environment** (e.g. *Valerion-UAT*) with these variables:

Variable	Initial value	Notes
base_url	http://localhost:7500	Gateway base URL
admin_secret	<ops-only value>	Only used to mint tokens; treat as secret
token	(leave blank)	Auto-populated by <code>/auth/token</code> request
coder_id	<your user UUID>	Used in submit payloads
report_id	(fill from queue response)	The report you are reviewing

4.2 Request 1 — Get a token

Method **POST**, URL `{{base_url}}/auth/token`. Headers: `X-Admin-Secret: {{admin_secret}}`, `Content-Type: application/json`. Body (raw JSON):

```
{
  "client_id": "postman-tester",
  "hospital_name": "Test Hospital",
  "daily_limit": 500
}
```

Then on the **Tests** tab, capture the JWT into the environment:

```
const data = pm.response.json();
pm.environment.set("token", data.access_token);
```

4.3 Request 2 — Review queue

Method **GET**, URL `{{base_url}}/api/review/queue?page=1&page_size=20`. Authorization tab: type *Bearer Token*, token `{{token}}`. Copy any id from the response into the `report_id` environment variable.

4.4 Request 3 — Get codes for a report

Method **GET**, URL `{{base_url}}/api/review/report/{{report_id}}/codes`, same Bearer auth. Note the id values — those are the `predicted_code_ids` you reference in the submit payload.

4.5 Request 4 — Submit corrections

Method **POST**, URL `{{base_url}}/api/review/report/{{report_id}}/submit`, Bearer auth, `Content-Type: application/json`. Body:

```
{
  "coder_id": "{{coder_id}}",
  "actions": [
    {
```

```

    "predicted_code_id": "<paste-from-step-4.4>",
    "action": "ACCEPT"
  },
  {
    "predicted_code_id": "<paste-from-step-4.4>",
    "action": "EDIT",
    "correct_code": "I10",
    "correct_description": "Essential (primary) hypertension",
    "reason": "Code in note is I10 not I11; HTN without heart involvement."
  },
  {
    "action": "ADD",
    "correct_code": "Z79.4",
    "correct_description": "Long term (current) use of insulin",
    "code_type": "secondary",
    "reason": "Patient on insulin per medication list – required per guidelines."
  }
]
}

```

Expected: 200 with corrections_written matching the number of EDIT + DELETE + ADD items. A 422 means a reason is under 20 characters or a required field is missing — fix and resend.

4.6 Quick troubleshooting

Symptom	Most likely cause
401 Unauthorized	Token expired or not pasted into {{token}}.
403 Invalid admin secret	X-Admin-Secret header missing or wrong on /auth/token.
404 Coder not found	coder_id env var is not a real User UUID in Postgres.
409 Report already approved	You already submitted for this report. Pick another from /queue.
422 reason >= 20 chars	Lengthen the reason text past 20 characters after trimming.
429 Too many requests	Daily limit hit — wait for reset or raise daily_limit.
503 Orchestrator unavailable	Orchestrator container is down. Check docker-compose ps.

5. Verification endpoints

The submit endpoints return a `correction_id` for each EDIT/DELETE/ADD result, but the response itself is the only proof the row landed. If your side needs to confirm what was actually written, audit a coder's history, or detect drift between your records and ours, use these two endpoints. Both are JWT-gated, same Bearer token as the review API.

5.1 GET /admin/corrections/{correction_id} (single)

Fetch one correction by its id. The id comes from the `correction_id` field on a submit-response result. Use this to verify a specific write landed and the stored fields match what you sent.

```
GET /admin/corrections/f0a91c3e-...
Host: <gateway-host>:7500
Authorization: Bearer <token>
```

Response 200:

```
{
  "id": "f0a91c3e-...",
  "report_id": "8d3c-...", // null for encounter flow
  "encounter_id": null, // null for report flow
  "action_type": "EDIT",
  "wrong_code": "I11.9",
  "wrong_code_description": "Hypertensive heart disease w/o heart failure",
  "correct_code": "I10",
  "correct_description": "Essential (primary) hypertension",
  "code_type": "primary",
  "sequence_pos": 1,
  "reason": "Code in note is I10 not I11; ...",
  "confidence_was": 0.78,
  "coder_id": "11111111-2222-3333-4444-555555555555",
  "reviewed_at": "2026-05-15T06:22:11+00:00",
  "ip_hash": "476c49ae8dbd09a6",
  "synced_to_qdrant": true
}
```

404 if no row with that id exists.

5.2 GET /admin/corrections (list with filters)

List corrections, ordered by `reviewed_at` descending. All filters optional, AND-combined; no filters = the most recent corrections globally (subject to limit).

Query param	Type	Description
<code>coder_id</code>	UUID	Filter to a specific coder's corrections
<code>report_id</code>	UUID	Filter to a specific report
<code>encounter_id</code>	UUID	Filter to a specific encounter
<code>action_type</code>	string	DELETE EDIT ADD
<code>since</code>	ISO-8601 ts	Only rows with <code>reviewed_at</code> >= this
<code>limit</code>	int	1–200, default 50
<code>offset</code>	int	Default 0, for pagination

```
GET /admin/corrections?coder_id=1111...&since=2026-05-15T00:00:00Z&limit=20
Host: <gateway-host>:7500
```

Authorization: Bearer <token>

Response 200:

```
{
  "items": [
    { "id": "f0...", "action_type": "EDIT", ... },
    { "id": "f1...", "action_type": "DELETE", ... },
    ...
  ],
  "total": 47,           // matches filters; not just this page
  "limit": 20,
  "offset": 0
}
```

5.3 How to use it — typical verification flows

Flow A — immediate verification after a submit.

1. POST /api/review/.../submit; receive 200 with a results[] array.
2. For each result that has a correction_id (i.e., EDIT/DELETE/ADD — not ACCEPT), call GET /admin/corrections/{correction_id}.
3. Compare action_type, correct_code, reason, coder_id against what you sent. Treat mismatch or 404 as a hard failure and alert ops — it means the submit-response lied or the row was mutated/deleted.

Flow B — periodic reconciliation.

1. On a schedule (nightly, hourly), call GET /admin/corrections?since=<last_run_ts>&limit=200, paging with offset until items.length < limit.
2. For each row, check that the matching record exists on your side with the same action_type and codes. Insert missing rows; alert on rows that exist on our side but not yours, or where fields diverge.
3. Persist the latest reviewed_at seen so the next run is incremental.

Flow C — per-coder audit.

Call GET /admin/corrections?coder_id=<uuid> to retrieve everything one coder has submitted. Useful for inter-rater analysis or investigating a flagged coder. Pair with action_type=DELETE to find every code a coder removed, etc.

5.4 Error responses

Status	When it happens
401	Missing / invalid / expired JWT.
404	(single only) No correction with that id.
422	Malformed `since`, or `action_type` not in DELETE EDIT ADD, or out-of-range limit/offset.
503	Orchestrator unreachable from gateway.

5.5 Tenancy caveat

These endpoints do **not** filter by the calling client's client_id (see §1.3). Any valid JWT can read any correction row. Treat the response as system-wide, not your-hospital-only. Per-tenant scoping lands with the broader tenancy work.

Appendix A — What happens behind the scenes

For each EDIT / DELETE / ADD action, the orchestrator invokes correction_writer.write_correction, which:

1. Inserts a row into Postgres `coder_corrections` with the wrong/correct codes, reason, `coder_id`, `sequence_pos`, `confidence_was`, and a hashed client IP. `synced_to_qdrant` starts as false.
2. Builds an embed string of the form "wrong_code wrong_desc | correct_code correct_desc | reason" (no PHI) and POSTs it to the RAG service `/write/correction_memory`.
3. RAG service embeds the text and upserts a point into the Qdrant `correction_memory` collection, using the Postgres correction UUID as the point id for traceability.
4. If the Qdrant upsert succeeds, the Postgres row is updated to `synced_to_qdrant = true`. Future pipeline runs query this collection in the pre-Agent-3 step, so the new correction immediately influences subsequent reports.

ACCEPT actions skip steps 1-4 and only write a ReviewedCode row — accepting the model's prediction is not a correction.